

Cuestionario de Evaluación · Implantación de Arquitecturas Web

1. Los tres elementos fundamentales del esquema de funcionamiento de los servicios web

El temario lo enuncia de forma literal: «el esquema de funcionamiento de los servicios web requiere de **tres elementos fundamentales**». Son estos:

#	Elemento	Función
1	Proveedor del servicio web	Es quien lo diseña, desarrolla e implementa y lo pone disponible para su uso, ya sea dentro de la misma organización o en público.
2	Consumidor del servicio	Es quien accede al componente para utilizar los servicios que éste presta.
3	Agente del servicio	Sirve como enlace entre proveedor y consumidor , a efectos de publicación, búsqueda y localización del servicio.

La idea que hay detrás es que, en la Web, el servicio ya no se presta y se consume de manera directa entre dos interlocutores: hay un **tercero** que se ocupa de publicarlo, buscarlo y localizarlo, y que es el que permite que el consumidor lo encuentre.

Otras lecturas del mismo esquema, por si el enunciado se mezclara con otro apartado.

En la unidad aparecen otros dos tríos que conviene no confundir con este:

- **Cliente, servidor y protocolo de comunicación.** Es como el temario define una aplicación web: «una aplicación cliente-servidor junto con un protocolo de comunicación previamente establecido». Aquí el nexo es **HTTP**, el protocolo más usado en la Web, que opera sobre el conjunto de protocolos TCP/IP.
- **Cliente, servidor web y base de datos.** Es el reparto de roles habitual: el navegador inicia el diálogo, el servidor web atiende las peticiones y sirve ficheros estáticos o las pasa a otros programas cuando el contenido es dinámico, y la base de datos almacena los registros que la capa de negocio consulta y modifica.

La respuesta al enunciado es la primera tabla; las otras dos sirven para demostrar que se domina el esquema completo.

2. Las tres capas de un modelo de arquitectura web

De forma genérica, la arquitectura web es un modelo compuesto de **tres capas**. Esta es la respuesta principal, porque es la única formulación del temario que trae **un ejemplo de software asociado a cada capa**, que es justo lo que pide el enunciado. El temario lo expresa en dos planos, y ambos aparecen en la misma unidad:

a) Capas según el despliegue y la tecnología

Capa	Contenido	Ejemplo de software asociado
Capa de base de datos	Toda la información que se pretende administrar mediante el servicio web: almacén, estructura y	MySQL, PostgreSQL

Capa	Contenido	Ejemplo de software asociado
	recuperación de los datos.	
Capa de servidores de aplicaciones web	Ejecuta las aplicaciones y gestiona las solicitudes y respuestas HTTP.	Apache, Tomcat, Resin
Capa de clientes	Punto de acceso del usuario al servicio.	Navegadores: Firefox, Internet Explorer, Opera

b) Capas según la responsabilidad funcional

Capa	Responsabilidad
Presentación	Se ocupa de la navegabilidad, la validación de los datos de entrada, el formateo de los datos de salida y la presentación de la web. Es la capa que se presenta al usuario.
Negocio	Recibe las peticiones del usuario, envía las respuestas y verifica que las reglas establecidas se cumplen.
Acceso a datos	Formada por los gestores de datos que almacenan, estructuran y recuperan los datos solicitados por la capa de negocio.

c) Formulación equivalente del propio temario

El temario vuelve a listar las tres capas en otro punto, con otra terminología pero el mismo reparto: **1. Navegador web. 2. Tecnología web dinámica (PHP, Java Servlets, ASP, etc.). 3. Base de datos.** Es la misma arquitectura expresada como **navegador, tecnología dinámica y base de datos**, y se puede citar como refuerzo.

La razón de separar así las capas está en los atributos que el temario destaca de una arquitectura web: **escalabilidad, separación de responsabilidades, portabilidad y utilización de componentes.** Gracias a esta separación lógica se puede llegar a una separación física de las capas (una máquina por capa mediante **middlewares**), que es la base de la escalabilidad vertical.

3. Diferencia entre PHP/ASP y JavaScript: ciclo de vida y lugar de ejecución

La diferencia técnica sustancial es **dónde se ejecuta el código**: en el **servidor** o en el **cliente**. De ahí se deriva todo lo demás (qué viaja por la red, qué se puede proteger y qué no).

Aspecto	PHP / ASP (lado del servidor)	JavaScript (lado del cliente)
Lugar de ejecución	En el servidor web o en el servidor de aplicaciones.	En el navegador del equipo cliente.
Cuándo se ejecuta	Tras cada petición: el servidor recibe la solicitud e invoca el intérprete o el motor del lenguaje antes de enviar nada.	Después de descargarse la página: cuando el HTML llega al navegador, el motor de JavaScript del navegador lo analiza y lo ejecuta.
Qué viaja por la red	El resultado ya generado (HTML con el contenido final).	El código fuente del script, que el cliente puede leer.
Acceso a datos	Puede consultar y modificar la base de datos y el sistema de archivos del servidor.	No puede acceder a la base de datos ni a ficheros del servidor; solo a lo que se le ha enviado. Para pedir más datos debe hacer una nueva petición (AJAX o fetch).

Aspecto	PHP / ASP (lado del servidor)	JavaScript (lado del cliente)
Ciclo de vida	Por petición: el script se ejecuta, produce la respuesta y termina. El estado entre peticiones se conserva solo con sesión.	Por página: desde la carga del documento hasta su cierre, manteniendo estado entre eventos.
Visibilidad y seguridad	El código no llega al cliente, por lo que se pueden ocultar datos y reglas de negocio.	El código es visible para cualquiera que abra el código fuente, por lo que no sirve para ocultar información sensible.

En el temario se ilustra así: «ASP (Active Server Pages): las Páginas Activas se ejecutan del lado del servidor», de modo que se forman los resultados que luego se mostrarán en el navegador de cada equipo cliente. Frente a ello, el temario explica que «cuando una página web llega al navegador es posible que también incluya algún programa o fragmento de código que se deba ejecutar. Ese código, normalmente en lenguaje JavaScript, lo ejecutará el propio navegador».

Ambos son lenguajes interpretados, pero **la diferencia no está en el lenguaje, sino en el punto de ejecución**: PHP y ASP generan la respuesta en el servidor; JavaScript solo da comportamiento a la página en el cliente.

4. Diferencias entre el «Modelo 1» y el «Modelo 2»

La evolución ha ido separando las responsabilidades que en el origen estaban mezcladas. El temario recorre cuatro modelos: Modelo 1, Modelo 1.5, Modelo 2 y Modelo 2X.

Aspecto	Modelo 1	Modelo 2
Base tecnológica	Modelo web CGI: ejecución de procesos externos al servidor web .	Patrón MVC , con la incorporación de un elemento controlador de la navegación de la aplicación.
Dónde vive cada responsabilidad	Presentación, negocio y acceso a datos se confunden en un mismo script (por ejemplo, en Perl o PHP).	Separadas lógicamente : el controlador dirige, las páginas JSP hacen de presentación y los JavaBeans encapsulan el modelo de negocio. Matiz del temario: los beans siguen incrustados en las páginas JSP , así que la separación es de responsabilidades, no todavía de ficheros.
Generación de la salida	El script produce el HTML que el navegador recibe como respuesta.	Las vistas JSP se procesan en el servidor y generan el HTML.
Unidad de código	Cada página es un programa completo que mezcla las tres capas.	El controlador es único y centraliza las peticiones; el código se reparte en clases y páginas.
Mantenimiento	Difícil: cualquier cambio afecta a toda la página y hay que repetir lógica en cada una.	Sencillo: cada pieza tiene una responsabilidad clara y se puede reutilizar y probar por separado.

Como escalón intermedio está el **Modelo 1.5**, propio de Java: aparecen las JSP y los servlets, la presentación recae en las páginas JSP y los beans incrustados en las mismas se encargan del negocio y del acceso a datos, pero todavía no hay un controlador que centralice la navegación. El **Modelo 2** añade ese controlador, que es precisamente el Servlet Controlador.

El temario completa la serie con el **Modelo 2X**, pensado para aplicaciones **multicanal** (atacables desde una PDA, un terminal de telefonía móvil o cualquier navegador HTML estándar), que usa

plantillas XSL para transformar los datos XML y publicar así la misma aplicación en distintos dispositivos.

La diferencia estructural clave es, por tanto, la **separación de responsabilidades**: en el Modelo 1 las tres capas conviven en un mismo script, mientras que en el Modelo 2 cada una tiene su componente y se comunican de forma controlada.

5. Los componentes de las plataformas LAMP y WISA

Una plataforma web se define por cuatro componentes: **sistema operativo**, **servidor web**, **gestor de bases de datos** y **lenguaje de programación interpretado** que controla las aplicaciones que corren en el sitio. Las combinaciones de esos cuatro componentes dan lugar a numerosas plataformas, pero dos destacan por su popularidad.

Componente	LAMP (software libre)	WISA (software propietario)
Sistema operativo	Linux	Windows
Servidor web	Apache	Internet Information Services (IIS)
Gestor de bases de datos	MySQL	SQL Server
Lenguaje de backend	PHP (aunque a veces se sustituye por Perl o Python)	ASP o ASP.NET

La diferencia de fondo es de **licencia**: LAMP trabaja enteramente con componentes de software libre y no está sujeta a restricciones propietarias, mientras que WISA está basada en tecnologías desarrolladas por la compañía Microsoft. Por eso el temario, en su apartado de instalación de Apache sobre Debian, menciona la ventaja de «montar una plataforma LAMP, por sus ventajas derivadas de las características del software libre».

Ambas plataformas se cierran con el mismo esquema de tres niveles: **navegador web**, **tecnología web dinámica** (PHP, Java Servlets, ASP) y **base de datos** encargada de almacenar de forma permanente y actualizada la información.

6. Balanceador de carga hardware tradicional frente a balanceador hardware HTTP

La escalabilidad horizontal consiste en «darle al sistema otra máquina de características similares y balancear la carga de trabajo mediante un dispositivo externo», y ese dispositivo puede ser de tres tipos. La diferencia entre el balanceador **hardware tradicional** y el **hardware HTTP** está precisamente en el **modo de operación** y en la **capacidad de inspección** del tráfico.

Los tres tipos de balanceador

Tipo	Modo de operación	¿Inspecciona el paquete HTTP?	¿Mantiene la relación usuario-máquina?	Velocidad
Balanceador software	Un servidor web como Apache con mod_jk (o	Sí, y además identifica la	Sí, por afinidad de sesión	La más lenta.

Tipo	Modo de operación	¿Inspecciona el paquete HTTP?	¿Mantiene la relación usuario-máquina?	Velocidad
	mod_proxy/mod_proxy_balancer) que redirige las peticiones HTTP entre las máquinas de la granja.	sesión del usuario guardando registro de qué máquina la atiende.	(sticky sessions).	
Balanceador hardware (tradicional)	Dispositivo que responde únicamente a algoritmos de reparto de carga (Round Robin, LRU) y conmuta circuitos ; reenvía la petición sin mirar su contenido.	No : no examina ni interpreta el paquete HTTP.	No , no puede garantizar que las peticiones de una misma sesión vayan a la misma máquina.	El más rápido.
Balanceador hardware HTTP	Dispositivo hardware que sí examina el paquete HTTP para decidir el nodo destino y mantiene la relación usuario-máquina .	Sí .	Sí .	Muy rápido: más que el software, algo menos que el hardware puro.

La diferencia concreta que pide la pregunta

- **Modo de operación.** El balanceador hardware tradicional trabaja de forma **opaca**, conmutando circuitos: aplica el algoritmo de reparto sin interpretar la conversación. El hardware HTTP, en cambio, **interpreta el protocolo de aplicación** para tratar cada petición individualmente, lo que le permite descartar, redirigir o reescribir antes de enviar.
- **Capacidad de inspección.** Al no examinar el paquete HTTP, el balanceador hardware tradicional **ignora quién es el usuario**. El hardware HTTP **lo identifica** y por eso puede mantener la relación usuario-máquina.
- **Consecuencia sobre el diseño de la aplicación.** Este es el efecto importante: el hardware tradicional «no garantiza el mantenimiento de la misma sesión de usuario en la misma máquina, [lo que] condiciona seriamente el diseño, dado que fuerza a que la información relativa a la sesión del usuario sea almacenada por el implementador del mismo, bien en cookies o bien en base de datos». El hardware HTTP evita esa penalización.
- **Costo.** El hardware HTTP es «mucho más rápido que los balanceadores software, pero algo menos que los hardware», y es una de las soluciones más aceptadas del mercado.

Si se quiere expresar esta diferencia en niveles OSI —**lectura propia, no del temario**—, el balanceador hardware tradicional opera a **capa 4** (conmutación de conexiones) y el hardware HTTP a **capa 7** (inspección del protocolo de aplicación). El balanceador software, que también inspecciona el HTTP, se sitúa igualmente en capa 7, pero con el coste de CPU de una máquina de propósito general.

7. Las tres categorías de módulos del servidor web Apache

El servidor está **estructurado en módulos**: cada módulo contiene un conjunto de funciones relativas a un aspecto concreto del servidor. El binario httpd contiene un conjunto de módulos

compilados cuya funcionalidad se activa o desactiva al arrancar el servidor. El temario los clasifica en tres categorías:

Categoría	Función general	Ejemplos
Módulos base	Se encargan de las funciones básicas del servidor.	core (funcionalidad siempre disponible en la distribución)
Módulos multiproceso	Encargados de la unión de los puertos de la máquina , es decir, de aceptar las peticiones y atenderlas.	MPM prefork, worker y event; directivas comunes en mpm_common
Módulos adicionales	Se encargan de añadir funcionalidad al servidor.	mod_alias, mod_rewrite, mod_ssl, mod_status, mod_proxy, mod_proxy_ajp, mod_proxy_balancer, mod_session

La estructura oficial de Apache HTTP Server 2.4 **se parece, pero no coincide**, con esta división: el índice separa los «**Core Features and Multi-Processing Modules**» (los módulos core, mpm_common y los MPM) del bloque «**Other Modules**», donde están los módulos adicionales. Las diferencias son que «Other Modules» no es una categoría funcional (conviven ahí mod_rewrite y mod_heartbeat) y que algunos módulos de uso básico, como mod_authz_core, mod_mime o mod_unixd, están igualmente en ese segundo bloque. **En un examen manda la clasificación del temario.** Además, mod_so carga como módulo compartido (DSO) el código ejecutable de módulos en el arranque o al reiniciar, y mod_unixd aporta la seguridad básica en plataformas Unix, donde se produce el cambio de privilegios a los procesos hijo.

En la práctica se controlan con a2enmod / a2dismod (Debian y Ubuntu) o con directivas LoadModule en apache2.conf, y son precisamente los que aparecen en la línea a2enmod proxy, a2enmod proxy_ajp y a2enmod proxy_balancer del temario.

En Debian y Ubuntu se habilitan con las utilidades a2enmod:

```
| sudo a2enmod proxy proxy_ajp proxy_balancer
```

8. Privilegios de root en el puerto 80 y atención posterior a los clientes

Por qué hace falta root

Los puertos inferiores al 1024 están reservados por convención a los procesos del sistema, y solo el usuario **root** (o un proceso que tenga esa capacidad) puede hacer **bind** sobre ellos. Por eso, si el puerto indicado en la directiva Listen es el habitual, **el 80, o cualquier otro por debajo del 1024, es necesario tener privilegios de usuario root (superusuario) para iniciar Apache**, de modo que pueda establecerse la conexión a través de esos puertos privilegiados.

Qué ocurre después del arranque

1. El proceso **principal**, el binario httpd, **arranca como root** y completa las tareas preliminares, entre ellas **abrir sus ficheros de log**.
2. Una vez hecho esto, **lanza varios procesos hijo**, que son los que hacen el trabajo de escuchar y atender las peticiones de los clientes.
3. **El proceso principal continúa ejecutándose como root, pero los procesos hijo se ejecutan con menores privilegios de usuario** (por ejemplo, www-data en Debian y Ubuntu, apache en Red Hat), según las directivas User y Group.

Por qué esto es seguro

El reparto de privilegios está pensado al revés de lo que parece a primera vista: el proceso que **no atiende** a los clientes es el único que conserva root, y los que **sí atienden** peticiones de usuarios arbitrarios funcionan sin privilegios. Así, una vulnerabilidad en un proceso hijo no entrega la máquina completa, y si un hijo muere, el proceso principal puede seguir funcionando y crear uno nuevo.

Conviene añadir un matiz honesto: los hijos se ejecutan todos con el **mismo** usuario sin privilegios, de modo que el aislamiento entre ellos es limitado; la protección real consiste en que ninguno trabaja como root.

La gestión del servicio se hace con el script de control `apachectl`, que interpreta los argumentos `start`, `restart` y `stop` y los traduce a las señales apropiadas para `httpd`. Y hay que recordar un detalle práctico: **Apache solo reconoce los cambios de configuración al reiniciar**, por lo que los cambios en `apache2.conf` o en los ficheros incluidos no surten efecto hasta entonces.

9. Los tres modos de despliegue de un contenedor de servlets

Un contenedor de servlets es el software que **maneja e invoca servlets por cuenta del usuario**. El temario los divide en tres modos según dónde se ejecute la máquina virtual Java respecto al servidor web.

Modo	Cómo funciona	Rendimiento	Escalabilidad y estabilidad
1. Stand-alone (independiente)	El contenedor de servlets es parte integral del servidor web . Es el caso en que se usa un servidor web basado en Java; el contenedor forma parte de este (por ejemplo, <code>JavaWebServer</code> , sustituido por <code>iPlanet</code>). Por defecto Tomcat trabaja en este modo.	—	—
2. Dentro de proceso (in-process)	Combinación de un plugin para el servidor web y una implementación de contenedor Java. El plugin abre una JVM dentro del espacio de direcciones del servidor web y el contenedor se ejecuta en ella. Si una petición debe ejecutar un servlet, el plugin toma el control de la petición y la pasa al contenedor usando JNI .	Alto: adecuado para servidores multihilo de un solo proceso.	Limitado: es el modo con mejor rendimiento, pero su escalabilidad es reducida.
3. Fuera de proceso (out-of-process)	Combinación de un plugin para el servidor web y una implementación de contenedor Java que se ejecuta en una JVM fuera del servidor web . El plugin y la JVM del contenedor se comunican con algún mecanismo IPC, normalmente sockets TCP/IP ; el plugin toma la petición y la pasa al contenedor.	Menor: el tiempo de respuesta no es tan bueno como en el anterior.	Mejor: a cambio mejora en otras cosas, sobre todo escalabilidad y estabilidad.

En la práctica, Tomcat puede utilizarse **como contenedor solitario** (principalmente para desarrollo y depuración) o **como plugin de un servidor web existente** (actualmente soporta Apache e IIS). Si se opta por las opciones 2 o 3, además hay que instalar un **adaptador de servidor web**.

10. Protocolo AJP: puerto habitual y ventajas frente a HTTP simple

El protocolo es **AJP (Apache JServ Protocol)**, implementado en Apache por el módulo `mod_proxy_ajp`, un módulo de soporte de AJP para `mod_proxy`. **Puerto habitual: el 8009**, que es el puerto de trabajo por defecto del conector AJP de Tomcat, aunque puede variarse en `conf/server.xml`. En Apache se declara como miembro del cluster con `BalancerMember ajp://localhost:8009`.

El temario lo describe como «un protocolo de comunicación interno y muy rápido que usa conexiones TCP persistentes», y señala que es el que se utiliza para comunicar Apache con Tomcat, «aunque podría ser utilizado HTTP».

Ventajas frente a HTTP simple

Ventaja	Explicación
Conexiones TCP persistentes	Es la ventaja principal. El conector AJP mantiene un conjunto de conexiones abiertas entre Apache y Tomcat en lugar de abrir y cerrar una por cada petición, ahorrando el coste del handshake TCP en cada petición.
Cabeceras pre-empacadas en formato binario	AJP no reenvía las cabeceras como texto plano, sino pre-empacadas en una estructura binaria compacta, donde los nombres más comunes vienen predefinidos como enteros y el resto como cadena con longitud. Evita analizar y volver a construir las cabeceras en cada salto, con lo que se reduce CPU y latencia.
Menor latencia	Al no gastar ciclos en serializar y deserializar cabeceras en texto, la respuesta es más rápida, sobre todo con muchas peticiones pequeñas.

Conviene ser precisos en dos puntos que suelen deformarse al explicar AJP:

- **El cuerpo de la petición sí se copia.** Viaja en un paquete de datos aparte, troceado en bloques de 8 KB (`packetSize` por defecto 8192), que el contenedor va pidiendo con peticiones `GET_BODY_CHUNK`. Lo que AJP evita es el **escaping** de caracteres en URLs y cookies, no la copia del cuerpo.
- **AJP no multiplexa.** Una vez que una conexión se asigna a una petición, no se reutiliza para ninguna otra hasta que termina el ciclo de esa petición. Por eso el conector mantiene varias conexiones abiertas: una por petición en vuelo, no una única conexión compartida.

Limitaciones y precauciones

- **No es un protocolo estándar para clientes:** está pensado para la comunicación interna entre el servidor web y el contenedor de servlets, no para que un navegador lo hable directamente.
- **El puerto 8009 no debe exponerse a Internet.** Tomcat moderno trae `secretRequired` con valor `true` por defecto, y eso no filtra peticiones: impide directamente que el **conector arranque** si no se define un `secret` compartido. Si se define, el secreto viaja en el campo `?secret` del paquete `AJP13_FORWARD_REQUEST`, y toda petición que no lo lleve se rechaza. En Apache hay que declararlo en el `ProxyPass` o en el `BalancerMember` (`secret=CLAVE`), lo que exige `httpd 2.4.42` o superior.

En el temario, la puesta en marcha consiste en cargar `proxy`, `proxy_ajp` y `proxy_balancer`, definir el `BalancerMember` con protocolo, IP y puerto, y usar `ProxyPreserveHost On` para mantener la cabecera `Host` original en lugar de reescribirla.

La configuración mínima en Apache, siguiendo el esquema del temario, es:

```
<VirtualHost *:80>
    ServerName www.ejemplo.com
    ProxyPreserveHost On
    ProxyPass      / http://localhost:8009/
    ProxyPassReverse / http://localhost:8009/
</VirtualHost>
```

11. Flujo de control de un Servlet Controlador en una arquitectura MVC

El Servlet Controlador es el **controlador de navegación** del Modelo 2. Todas las peticiones de la aplicación pasan por él, y el flujo hasta generar la vista final es el siguiente:

1. **Llega la petición.** El navegador envía una petición HTTP (GET o POST) a una URL de la aplicación.
2. **Resolución en el contenedor.** El contenedor de servlets (Tomcat) recibe la petición y la resuelve contra el **descriptor de despliegue** WEB-INF/web.xml, que mapea ese patrón de URL con el Servlet Controlador declarado mediante <servlet>, <servlet-name>, <servlet-class> y <servlet-mapping>.
3. **Instancia y llamada.** El contenedor crea una instancia del servlet (o reutiliza una del **pool**) y llama a su método service(), que delega en doGet() o doPost() según el método HTTP. El contenedor gestiona el ciclo de vida y la ejecución multihilo.
4. **Recogida de parámetros.** El controlador extrae los datos de entrada con request.getParameter() y, si hace falta, del objeto de sesión.
5. **Llamada al modelo.** El controlador delega la lógica de negocio en el **modelo**: objetos JavaBean o clases de negocio que aplican las reglas y acceden a los datos de la capa de acceso a datos. El controlador **no debe contener reglas de negocio**; solo coordina.
6. **Preparación de los datos.** El modelo devuelve el resultado y el controlador lo coloca en el **ámbito de la petición** (request.setAttribute()) o de la sesión (session.setAttribute()), para que la vista pueda leerlo.
7. **Selección de la vista.** El controlador decide cuál es la siguiente vista y devuelve su **nombre lógico**, no su ruta física. Este desacoplo es una ventaja del patrón: cambiar el nombre de un fichero JSP no obliga a cambiar el código Java.
8. **Reenvío interno a la vista.** El controlador **reenvía** (forward) con RequestDispatcher a la página JSP. El reenvío es interno al servidor, de modo que la URL del navegador no cambia.
9. **Generación del HTML.** Tomcat compila la JSP a servlet mediante **Jasper** (el propio compilador que incluye Tomcat) y la ejecuta en el servidor, usando los atributos preparados con EL o JSTL para componer el HTML final.
10. **Envío de la respuesta.** El servidor devuelve el HTML al navegador, que lo pinta y ejecuta el JavaScript del cliente que lo pinte.
11. **Tareas transversales.** Los **filters** pueden actuar antes y después del controlador (autenticación, logging, control de acceso) sin mezclarse con el MVC, y el controlador se encarga de la gestión de errores y del control de acceso.

En resumen, el Servlet Controlador **recibe la petición, extrae los datos, llama al modelo, coloca el resultado en el ámbito correspondiente, elige la vista y reenvía a ella**; la vista solo pinta y el modelo solo aplica reglas y accede a los datos. Este es el patrón **Front Controller**,

que centraliza todo el flujo en un único punto de entrada; una implementación conocida de ese patrón en la práctica es el `DispatcherServlet` de Spring.

12. Round Robin frente a LRU en servidores web balanceados

Round Robin

Reparte las peticiones **de forma cíclica y sin memoria**: petición 1 al nodo A, petición 2 al B, petición 3 al C, petición 4 otra vez a A, y así sucesivamente.

- Es **sin estado (stateless)**: no necesita recordar nada de las peticiones anteriores.
- Es **determinista y muy barato**: basta un contador incremental, sin bloqueos.
- Pero **ignora la carga real**: si el nodo C está saturado y el A está libre, Round Robin sigue mandando trabajo por igual. Solo reparte **peticiones**, no **trabajo**.

LRU (Least Recently Used)

El balanceador **recuerda en qué máquina se ha atendido a cada usuario** y, ante una petición, elige la máquina **cuyo último uso es más antiguo**. Es decir, LRU no reparte por carga: reparte por **recencia del último uso**.

- **Ventaja**: aporta **afinidad de sesión** sin interpretar el contenido de la petición, porque se apoya en la identidad del usuario.
- **Inconveniente**: es **más complejo y más caro**, porque obliga a mantener y actualizar una tabla de estado en cada petición.

Conviene ser exacto con una cosa: **LRU no reparte por carga ni por coste de la petición**. Un nodo con una petición larga y reciente sigue apareciendo como «usado» aunque esté saturado, mientras que otro con peticiones cortas puede parecer libre. Para repartir por carga real existen otros métodos, como **least connections** o **least response time**.

La problemática con peticiones concurrentes

El problema de fondo es que **el estado que LRU necesita choca con la propia naturaleza del balanceador**, y la concurrencia lo agrava:

1. **Necesita identificar al cliente, y el balanceador tradicional no puede**. Este es el punto clave: para saber a qué usuario pertenece una petición hay que poder distinguirla. Un balanceador que, como el hardware tradicional, «no examina ni interpreta el paquete HTTP» solo ve direcciones IP y puertos, de modo que **no puede garantizar la afinidad de sesión**. Por eso el temario reserva el LRU para el hardware HTTP y no para el conmutador puro.
2. **Dos peticiones simultáneas de la misma sesión**. Con una sesión abierta, el orden en que se registran los «últimos usos» es ambiguo: si llegan a la vez dos peticiones del mismo usuario, ambas pueden acabar en nodos distintos y **romper** la `HttpSession`. Es la traducción directa de la pérdida de afinidad.
3. **El estado convierte al balanceador en un punto único de fallo**. Su tabla usuario-máquina es crítica. Con varias instancias del balanceador esas tablas divergen y hay que sincronizarlas; Apache documenta los fallos típicos de este tipo de tablas: «distribución de carga desigual si los clientes están ocultos tras **proxys**, errores de afinidad con direcciones IP dinámicas y pérdida de afinidad si la tabla se desborda».

4. **Un nodo ocupado por una petición larga no se libra de trabajo.** LRU solo mira cuándo se usó el nodo por última vez, no cuánto le queda por terminar, de modo que penaliza a los nodos que están precisamente saturados.

La condición de carrera clásica —dos lecturas simultáneas del mismo estado antes de que ninguna lo actualice— solo sería defendible si el reparto con estado se implementase **en software con estado compartido**; en un aparato hardware dedicado no hay hilos ni copias locales de la tabla, y ese argumento no se sostiene.

En el lado de Apache el mismo problema se resuelve con los métodos de reparto de `mod_proxy_balancer`: `byrequests` (cuenta de peticiones, equivalente a un Round Robin), `bybusyness` (el nodo con menos peticiones pendientes, que sí tiene en cuenta la carga), `bytraffic` y `heartbeat`. Los tres últimos requieren mantener el estado de carga de los workers, y por eso el balanceador software es necesariamente más lento que un dispositivo dedicado.

Conclusión: Round Robin es simple, barato y escalable, pero reparte a ciegas y no tiene en cuenta la carga. LRU aporta la afinidad de sesión, pero para ello necesita identificar al usuario y mantener estado, lo que en un balanceador que no inspecciona el HTTP no es viable y, con peticiones concurrentes, compromete la afinidad. La coherencia del sistema depende de que ambas cosas encajen: en la práctica, o se usa un balanceador que sí inspecciona el HTTP, o la información de sesión se almacena fuera (cookies, base de datos o almacén compartido).

13. JNI: papel en la integración de componentes y su impacto en la estabilidad

Qué papel cumple

JNI (Java Native Interface) es la **interfaz estándar de la JVM** que permite a código nativo (C o C++) invocar y manipular código Java, y viceversa. Su papel en la integración de componentes es precisamente eso: **un puente entre el mundo Java y el mundo nativo**. Conviene precisar dos cosas: el **espacio de direcciones compartido** es una propiedad del **modo de despliegue dentro de proceso** (el **plugin** carga la JVM como una biblioteca más del proceso del servidor web), no de JNI en sí; y JNI, aunque es la vía estándar, no es la única técnica posible de interoperabilidad. Su papel en la integración de componentes es precisamente eso: **un puente entre el mundo Java y el mundo nativo dentro del mismo espacio de direcciones**.

En el temario aparece justo en el modo de despliegue **dentro de proceso** de los contenedores de servlets: «el plugin del servidor web abre una JVM (Máquina Virtual Java) dentro del espacio de direcciones del servidor web y permite que el contenedor Java se ejecute en él. En el caso de que una petición debiera ejecutar un servlet, el plugin toma el control sobre la petición y la pasa al contenedor Java (usando JNI)». Es decir, la JVM se integra en el proceso del servidor web y el traspaso de peticiones se hace a través de JNI.

El mecanismo se apoya en un **puntero de interfaz** que el nativo recibe como `JNIEnv`, con el que puede llamar a los métodos JNI para obtener objetos, invocar métodos Java, crear excepciones, liberar referencias, etc.

Impacto sobre la estabilidad global del servidor web

El impacto es **muy alto en sentido negativo**, y es la contrapartida directa de esa integración. Las razones:

1. **No hay aislamiento.** El código nativo se ejecuta **en el mismo espacio de direcciones** y dentro del mismo proceso que el servidor web. No hay fronteras de memoria que protejan al servidor de un fallo del componente nativo: **un fallo grave o una caída (segfault) del código nativo se lleva por delante al proceso completo.**
2. **No hay excepciones que lo protejan.** La JVM no puede atrapar los errores de memoria que ocurran en C: son **segmentation faults** o corrupciones de memoria, no excepciones Java. La especificación de JNI lo dice de forma explícita: «el programador no debe pasar punteros ilegales o argumentos del tipo incorrecto a funciones JNI. Hacerlo podría tener consecuencias arbitrarias, incluyendo un **estado del sistema corrupto o el fallo de la VM**».
3. **Fugas de referencias y corrupción silenciosa.** Si el código nativo no libera correctamente las referencias globales o locales, o las usa desde otro hilo, se puede producir una fuga de memoria o un comportamiento no determinista. La propia especificación advierte de que el puntero de interfaz «solo es válido en el hilo actual» y de que la actualización simultánea de un array desde varios hilos produce resultados no deterministas.
4. **Sin red de seguridad frente a la concurrencia.** La VM no aísla los errores del código nativo: si dos hilos usan a la vez un mismo recurso nativo, el resultado no es una excepción de Java sino corrupción de datos o un fallo del proceso. Por eso la especificación exige compilar las bibliotecas nativas con soporte multihilo (-mt, -D_REENTRANT).
5. **Menor escalabilidad.** El temario ya lo apunta al describir el modo dentro de proceso: «proporciona un buen rendimiento pero está limitado en escalabilidad».

Conclusión: JNI da acceso al código nativo sin capas de traducción, lo que permite **reutilizar librerías nativas ya existentes** (no acelerar por sí sola: la propia especificación advierte de que el acceso a objetos Java a través de funciones JNI tiene más coste que el acceso directo a las estructuras de datos de C), pero **sacrifica la estabilidad**. Un servidor web con componentes nativos y JNI está expuesto a que un solo fallo de ese componente tumbé el servicio completo. En la arquitectura del temario, por eso, el modo **fuera de proceso** sacrifica tiempo de respuesta a cambio de obtener mejores resultados en «escalabilidad y estabilidad»: si la JVM nativa se cae, al menos Apache sobrevive.

14. Replicación de sesión en un cluster de servidores de aplicaciones

Principio de funcionamiento

Una sesión HTTP vive **en memoria** del servidor que la creó (por ejemplo, el objeto HttpSession en Java). En un cluster, cada petición puede ser servida por una máquina distinta, de modo que la máquina que recibe una petición puede no tener la sesión del usuario.

La replicación de sesión consiste en **mantener una copia del estado de la sesión compartida entre todos los nodos del cluster**. Así, sea cual sea la máquina que sirva la petición HTTP, tendrá acceso a la sesión del usuario. Es lo que el temario describe al final del apartado de escalabilidad: «el cluster, mediante el mecanismo de replicación de sesión, garantiza que sea cual sea la máquina que sirva la petición HTTP, tendrá acceso a la sesión del usuario (objeto HttpSession en Java). Este tipo de sistemas, debido precisamente a la replicación de sesión, suele presentar problemas de rendimiento».

En Tomcat, el elemento <Cluster> activa tres funciones: la **replicación de sesiones**, la **replicación de atributos del contexto** y el **despliegue de archivos WAR en todo el cluster**.

Por debajo:

- Un **Session Manager** que decide qué se envía. El de por defecto es el `DeltaManager`, que envía solo los **cambios** (deltas) de la sesión en lugar de la sesión completa, y está acoplado a `SimpleTcpCluster`.
- Un **Channel**, denominado **Tribes**, que aporta la capa de comunicación y la lógica de **pertenencia** (qué nodos forman el cluster).
- Un **Valve**, que registra cuándo entran y salen las peticiones y usa esa información para decidir **cuándo** replicar: la replicación ocurre **siempre al final de una petición**, no en cada modificación.

El atributo `channelSendOptions` es un buen ejemplo de las decisiones de diseño: con `asynchronous` (valor por defecto, 8) los mensajes se colocan en una cola y los envía otro hilo, lo que da más rendimiento pero puede hacer que dos actualizaciones se procesen en el nodo receptor **en un orden distinto al de envío**; con `use_ack` (2) se exige confirmación de **recepción**, y con `synchronized_ack` (4) confirmación de **recepción y procesamiento** —además, esta última solo tiene efecto si `use_ack` está activo—, a cambio de más tráfico y más latencia.

Por qué se convierte en un cuello de botella

1. **Coste $O(n)$ por escritura.** Cada vez que una sesión cambia, hay que **serializar** el estado y **enviarlo a los otros $n-1$ nodos**. Cuantos más nodos se añaden para escalar, más crece el tráfico de replicación, hasta llegar a consumir más red que la propia aplicación. **Añadir máquinas empeora el rendimiento** justo cuando se necesita mejorarlo.
2. **Duplicación de memoria en todas las máquinas.** Cada nodo guarda las sesiones de **todos** los usuarios, no solo las suyas. La memoria por nodo crece con el total de usuarios del sistema, y llega un punto en que la memoria es el límite antes que la CPU.
3. **Latencia añadida y contención.** La serialización y la transmisión consumen CPU y red en la ruta crítica de la respuesta. Además, si la aplicación modifica la sesión en muchos puntos, se generan muchos mensajes pequeños, que saturan el canal y generan contención.
4. **Coste de serialización.** El estado de la sesión (atributos Java arbitrarios) debe serializarse; si contiene objetos no serializables, aparecen problemas adicionales de rendimiento y de corrección.
5. **Impacto en la disponibilidad.** Si un nodo cae, las sesiones que tenía ahora están en los demás (ventaja), pero durante la resincronización se pueden perder cambios no confirmados.

Alternativas

- **Almacén externo compartido** para las sesiones (Redis o Memcached) en lugar de replicación: una sola fuente de verdad, sin tráfico $O(n)$. Es la opción más usada en despliegues reales.
- **Afinidad de sesión (sticky sessions):** el balanceador mantiene al usuario en el mismo nodo y se replica solo como respaldo ante fallos. Reduce mucho el tráfico, pero pierde el equilibrio de carga y complica el **failover**.
- **Almacén de sesiones configurable en Apache:** el balanceador y el módulo `mod_session` permiten definir el almacén de sesiones (cookie, base de datos, DBD).
- **Diseño sin sesión (stateless):** se evita guardar estado en el servidor (por ejemplo, JWT con la información necesaria en el propio token), de modo que cualquier nodo puede atender cualquier petición y la escalabilidad horizontal es total.

La replicación se activa declarando el Cluster en conf/server.xml, y el secret del conector AJP debe coincidir con el configurado en Apache:

```
<Connector port="8009" protocol="AJP/1.3" secretRequired="true" secret="CLAVE"/>

<Cluster className="org.apache.catalina.ha.tcp.SimpleTcpCluster"
  channelSendOptions="6">
  <Manager className="org.apache.catalina.ha.session.DeltaManager"/>
</Cluster>
```

Fuentes consultadas

- **Temario de la unidad:** Implantación_de_arquitecturas_web.pdf (apartados 1.4 Arquitecturas web. Modelos; 1.5 Plataformas web libres y propietarias; 1.6 Escalabilidad; 2. Servidor web Apache; 2.2 Iniciar Apache; 3.1 El servidor de aplicaciones Tomcat; 3.1.2 Iniciar Tomcat; 4.2 Despliegue de aplicaciones con Tomcat; 4.3 Descriptor de despliegue).
- Apache HTTP Server 2.4 — índice de módulos: <https://httpd.apache.org/docs/2.4/mod/>
- Apache HTTP Server 2.4 — módulo core: <https://httpd.apache.org/docs/2.4/en/mod/core.html>
- Apache HTTP Server 2.4 — mod_proxy: https://httpd.apache.org/docs/2.4/en/mod/mod_proxy.html
- Apache HTTP Server 2.4 — mod_proxy_ajp: https://httpd.apache.org/docs/2.4/en/mod/mod_proxy_ajp.html
- Apache HTTP Server 2.4 — mod_proxy_balancer: https://httpd.apache.org/docs/2.4/en/mod/mod_proxy_balancer.html
- Apache Tomcat — conector AJP: <https://tomcat.apache.org/tomcat-11.0-doc/config/ajp.html>
- Apache Tomcat — el objeto Cluster (replicación de sesiones): <https://tomcat.apache.org/tomcat-11.0-doc/config/cluster.html>
- Apache Tomcat — índice de documentación: <https://tomcat.apache.org/tomcat-11.0-doc/index.html>
- Oracle — Java Native Interface Specification, capítulo 2 «Design Overview»: <https://docs.oracle.com/en/java/javase/21/docs/specs/jni/design.html>
- MDN — JavaScript en el navegador: <https://developer.mozilla.org/es/docs/Web/JavaScript>